

Pertinence des langages de programmation à l'ère de l'IA générative

TL;DR

Question de départ : L'IA générative (ChatGPT, Copilot, Claude) va-t-elle rendre certains langages de programmation obsolètes ? Les langages dont le code est "facilement générable par LLM" sont-ils en déclin accéléré ?

Méthodologie : Nous avons construit un panel mensuel de 21 165 observations couvrant 60 langages sur 2004-2024, à partir de 10 sources (Stack Overflow, TIOBE, GitHub). Un pipeline ML complet (LogReg, XGBoost, LightGBM, SHAP) prédit le déclin relatif d'un langage sur différents horizons temporels. Le tout est complété par une analyse de sentiment NLP (Twitter/RoBERTa) et un suivi des prédictions d'entrepreneurs tech.

Découverte principale : L'hypothèse "l'IA tue certains langages" n'est **pas confirmée** par les données. Notre score `ai_susceptibility` — mesurant la facilité de génération LLM par langage — arrive au rang 17/21 en importance SHAP. Les facteurs réellement prédictifs du déclin sont l'âge du langage, son niveau d'abstraction, et s'il est compilé. Ce n'est pas l'IA qui décide quels langages déclinent : ce sont des dynamiques structurelles et de marché bien plus anciennes.

1. Contexte et hypothèse

Depuis la sortie de ChatGPT (novembre 2022) et de GitHub Copilot, de nombreux leaders de l'industrie ont prédit la fin du code manuel :

- **Jensen Huang** (NVIDIA, fév. 2024) : *"It is our job to create computing technology such that nobody has to program."*
- **Dario Amodei** (Anthropic, mars 2025) : *"AI could write 90% of code in 3 to 6 months."*
- **Andrej Karpathy** (janv. 2023) : *"The hottest new programming language is English."*
- **Sam Altman** (OpenAI, janv. 2024) : *"AI coders will surpass humans by end of 2025."*

En parallèle, les prédictions spécifiques à des langages se sont avérées plus fiables : - **Linus Torvalds** : *"Let's wait 10 years"* — C toujours dominant dans le kernel. - **Anders Hejlsberg** : TypeScript en croissance (+66% de contributeurs en 2024). - **Mark Russinovich** (Microsoft Azure) : Rust en croissance comme remplaçant de C/C++.

Sur nos 15 prédictions suivies, 9 se sont réalisées (60%), 3 sont en cours, et 3 ne se sont pas réalisées. Les prédictions les plus fiables viennent des créateurs de langages spécifiques, pas des CEOs généralistes.

Hypothèse testée : Si l'IA générative accélère le déclin de certains langages, ceux dont le code est le plus facilement générable par LLM (JavaScript, PHP, Ruby — code web répétitif) devraient décliner plus vite que ceux qui résistent à la génération (Rust, C, Haskell — code système/formel).

2. Données et méthodologie

2.1 Sources de données

Source	Type	Couverture	Granularité
Stack Overflow Developer Survey	% d'usage auto-déclaré	2017-2024 (6 années)	Annuel
TIOBE Index	Part de marché (requêtes moteurs)	Jul 2004 - Dec 2024	Mensuel
GitHub Issues + PRs	Activité open-source	Q3 2011 - Q4 2022	Trimestriel
GitHub Daily Trending	Repos tendances	Nov 2024 - Oct 2025	Quotidien
Twitter Entity Sentiment	Sentiment général	~75 000 tweets	Ponctuel

2.2 Construction du panel

- **Panel unifié mensuel** : 12 148 lignes, 80 langages, 3 sources principales
- **Normalisation des noms** : 20+ variantes mappées (ex: "Visual Basic (.Net)" → "VB.NET"), HTML/CSS exclus
- **Features temporelles** : delta 1/3/6/12 obs, moyennes mobiles, volatilité 12 mois, momentum 6 mois, distance au pic, rang
- **Target** : déclin binaire (variation relative < -15% sur N observations)
- **Panel final avec targets multi-horizons** : 21 165 lignes x 38 colonnes, 60 langages

2.3 Stratégie ML

- **Split temporel strict** : train $\leq$ Dec 2020, val Jan 2021 - Jun 2022, test $\geq$ Jul 2022
- **Modèles testés** : LogReg L2, Random Forest, XGBoost, LightGBM
- **Évaluation** : F1 macro, ROC-AUC, PR-AUC, block bootstrap CI 95% par langage
- **Vérification anti-fuite** : toutes les features backward-looking, targets via shift(-N)

3. Résultats EDA

3.1 Tendances post-LLM (2022-2024)

Les plus fortes hausses relatives post-ChatGPT : - **Rust** : +187% relatif (de ~4.5% à ~12.6%) - **TypeScript** : +78.5% - **Go** : +71.9%

Les plus fortes chutes : - **VB.NET** : -35.9% - **PHP** : -30.1% - **Ruby** : -28.6% - **Swift** : -25.8%

3.2 Cohérence inter-sources

La corrélation entre Stack Overflow et GitHub atteint 0.861, confirmant que les deux sources racontent la même histoire. Le "point de rupture" 2022 est visible : les changements s'accélérent après ChatGPT, mais ils suivent des tendances structurelles préexistantes.

4. Résultats du modeling

4.1 Performances comparées

Modèle	F1 macro test	ROC-AUC	PR-AUC	CI 95% bootstrap
Baseline naïve (majorité)	0.459	—	—	—
Baseline métier (momentum+delta)	0.451	—	—	[0.403, 0.496]
LogReg L2	0.628	0.781	0.434	[0.523, 0.714]
LightGBM	0.543	0.693	0.281	—
XGBoost	0.550	0.685	0.298	—
Random Forest	0.521	0.695	0.304	—

Le LogReg L2 bat les deux baselines de +0.17 F1 (significatif, CIs non chevauchants). Les modèles à arbres sur-apprennent massivement (XGBoost : CV F1=0.74 vs test F1=0.54, gap +0.20).

4.2 Le twist : ai_susceptibility n'est pas prédictif

C'est le résultat le plus important de l'étude. Notre score `ai_susceptibility_score` — combinant une composante manuelle (facilité de génération LLM) et une composante data-driven (taux d'adoption IA par langage dans les SO 2023-2024) — arrive seulement au **rang 17 sur 21 features** en importance SHAP.

Top 5 features prédictives du déclin :

Rang	Feature	Interprétation
1	language_age	Les vieux langages déclinent plus
2	level_num	Langages bas niveau = plus stables
3	is_compiled	Compilé = moins de déclin
4	distance_to_peak	Loin du pic historique = déclin probable
5	usage_pct	Usage actuel élevé = inertie

4.3 Pourquoi l'hypothèse IA ne se confirme pas ?

Quatre explications complémentaires :

- Effet de second ordre** : L'IA aide à *écrire* du JavaScript, mais cela ne réduit pas la *demande* pour JavaScript. Au contraire, si le code est plus facile à produire, la demande pour le langage pourrait augmenter (paradoxe de Jevons).
 - Lenteur des cycles d'adoption** : Un langage ne disparaît pas en 2 ans. COBOL est prédit mort depuis 1990. Les entreprises ont des millions de lignes de code en production — la migration coûte plus cher que le maintien.
 - Biais de l'hypothèse** : Nous avons supposé que "générable par LLM = vulnérable". Mais les langages les plus générables (Python, JavaScript) sont aussi les mieux documentés, les plus communautaires, les plus demandés. La "générabilité" est un proxy de la popularité, pas de la vulnérabilité.
 - Fenêtre d'observation trop courte** : L'ère LLM date de fin 2022. Avec seulement ~2 ans de recul, les effets structurels n'ont peut-être pas eu le temps de se matérialiser dans les métriques de popularité.
-

5. Sentiment communautaire et avis d'experts

5.1 Sentiment Twitter (RoBERTa)

Sur 5 000 tweets échantillonnés et analysés via `cardiffnlp/twitter-roberta-base-sentiment-latest`, seuls 13 tweets ont pu être attribués à un langage spécifique (6 langages détectés). Le dataset Twitter est **générique** (entités comme Google, FIFA, Amazon) et non spécifique aux développeurs.

Limitation majeure : Ce signal est trop faible pour être statistiquement exploitable. Le sentiment est donc **exclu** du score de pertinence final (poids = 0).

5.2 Track record des prédictions célébrités

Catégorie	Score	Exemples
Prédictions réalisées (9/15)	1.0	Pichai (Google 25% AI code), Torvalds (C stable), Hejlsberg (TS growth)
En cours (3/15)	0.5	Karpathy (English = programming), Andreessen (AI eating software)
Non réalisées (3/15)	0.0-0.25	Huang (don't learn to code), Amodei (90% code en 6 mois)

Crédibilité globale : 0.72/1.00

Observation : les prédictions les plus fiables sont celles des créateurs de langages (Torvalds, Hejlsberg, van Rossum), pas celles des CEOs généralistes. Les prédictions quantitatives et chronologiques ("90% en 6 mois") se réalisent rarement dans les délais annoncés.

Disclaimer : 15 citations constituent un échantillon anecdotique, pas une analyse statistique. Cette section est illustrative.

6. Score de pertinence final

6.1 Méthodologie du score

Vu que `ai_susceptibility_score` n'est pas prédictif (rang 17/21 SHAP), il est **exclu** du score final. De même, le sentiment Twitter est exclu car seuls 13 tweets sur 5 000 ont pu être attribués à un langage (signal inexploitable). La pondération révisée :

```

pertinence(langage) =
    0.50 * (1 - prob_décline_modèle)      # prédiction ML
+ 0.25 * usage_actuel_normalisé          # taille du marché
+ 0.00 * sentiment_communauté           # exclu (13 tweets seulement)
+ 0.15 * momentum_normalisé              # tendance récente
+ 0.10 * stabilité_structurelle           # 1 - volatilité

```

Ce choix est délibéré : inclure une feature non prédictive ou un signal statistiquement inexploitable dans le score final biaiserait les recommandations sans apporter de valeur informative.

6.2 Top 15 — Langages à apprendre en 2026

Rang	Langage	Score	Forces
1	Python	0.898	#1 usage, momentum maximal, écosystème IA/ML
2	R	0.613	Niche solide (stats/data science), stable
3	C#	0.596	Écosystème .NET, usage entreprise solide
4	TypeScript	0.578	Superset JS dominant, forte croissance
5	JavaScript	0.576	Incontournable web, énorme écosystème
6	Kotlin	0.568	Android officiel, moderne, stable
7	PowerShell	0.550	Niche DevOps/admin, très stable
8	Go	0.532	Cloud-native dominant, syntaxe simple
9	Rust	0.531	Forte croissance, adoption Microsoft/Linux
10	Swift	0.531	Écosystème Apple, bonne stabilité
11	Java	0.520	Massif en entreprise, mais momentum faible
12	Dart	0.512	Flutter/multi-plateforme, en croissance
13	Lua	0.506	Niche jeux/embarqué, stable
14	VBA	0.485	Niche Office/macro, stable par inertie
15	Julia	0.476	Calcul scientifique, en croissance

6.3 Bottom 10 — Langages à éviter pour un débutant

Rang	Langage	Score	Risques
51	ActionScript	0.333	Flash mort, aucun écosystème
52	PHP	0.327	En déclin relatif constant
53	COBOL	0.324	Legacy mainframe, communauté mourante
54	Objective-J	0.317	Quasi-mort, remplacé par frameworks modernes
55	C/C++ (TIOBE)	0.304	Score bas car TIOBE les combine, haute volatilité
56	Fortran	0.300	Niche HPC, aucune croissance
57	C	0.278	Stable en absolu mais volatile, usage en baisse relative
58	Delphi/Object Pascal	0.273	Legacy, communauté réduite
59	VB.NET	0.268	Microsoft l'a abandonné, en déclin constant
60	Perl	0.257	Remplacé par Python depuis 15 ans

Important — Comment lire ce ranking

Ce classement mesure la *pertinence pour un développeur qui choisit un langage à apprendre en 2026*, pas la valeur intrinsèque du langage. Un langage en bas du classement n'est pas "mauvais" — il peut être indispensable dans son domaine :

- **C** (rang 57) reste incontournable pour le développement système, embarqué et kernel. Son score bas reflète une haute volatilité dans nos données et un usage en baisse *relative*, pas une obsolescence.
- **PHP** (rang 52) propulse encore ~75% du web (WordPress, Laravel). Son déclin est relatif à d'autres langages, pas absolu.
- **VBA** (rang 14) reste le seul moyen d'automatiser Excel/Office dans beaucoup d'entreprises.
- **Fortran** (rang 56) est encore utilisé en calcul scientifique haute performance et en météorologie.

Le score pénalise les langages volatils ou en baisse relative, ce qui désavantage les langages matures à usage stable mais non croissant. Pour un choix de carrière, croisez ce ranking avec les offres d'emploi de votre secteur.

6.4 Visualisation quadrant

Voir `fig_14_pertinence_quadrant.png` : axe X = usage actuel, axe Y = probabilité de déclin, couleur = sentiment communauté, taille = score de pertinence.

7. Limitations méthodologiques

Cette section est **critique** pour l'honnêteté scientifique du projet.

1. **Petit nombre de "vrais" déclin observés** : Sur 60 langages, seuls ~12 montrent un déclin significatif. Le modèle apprend surtout à distinguer les langages stables des quelques langages en chute — une tâche plus facile qu'elle n'y paraît.
 2. **Source TIOBE controversée** : L'indice TIOBE mesure la popularité via les requêtes de moteurs de recherche, une méthodologie critiquée par la communauté. Nos résultats en dépendent partiellement (7 134 lignes sur 21 165).
 3. **Sentiment Twitter générique** : Le dataset Twitter porte sur des entités générales (Google, FIFA, Amazon), pas sur des discussions de développeurs. Seuls 13 tweets sur 5 000 échantillonnés ont pu être attribués à un langage de programmation. Ce signal est anecdotique et a été exclu du score final.
 4. **15 citations célébrités = anecdotique** : Le credibility score est illustratif, pas statistique. 15 prédictions ne constituent pas un échantillon représentatif.
 5. **Effet IA potentiellement trop récent** : L'ère LLM date de fin 2022. Avec seulement ~2 ans de recul, les effets structurels sur les langages n'ont peut-être pas eu le temps de se matérialiser dans les métriques de popularité.
 6. **Gaps temporels SO** : Pas de Stack Overflow Survey en 2019 ni 2021. Les variations entre 2018→2020 et 2020→2022 couvrent 2 ans chacune, pas 1 an. Géré via `is_interpolated_monthly` mais imparfait.
 7. **Modèle entraîné majoritairement sur données pré-LLM** : Le train set (≤ 2020) ne contient aucune observation de l'ère ChatGPT. Le modèle extrapole des patterns pré-IA vers une ère post-IA, ce qui est fondamentalement risqué.
 8. **Overfitting des modèles non-linéaires** : XGBoost et LightGBM montrent un écart CV/test de +0.18 à +0.20 en F1, signe que le signal est faible et bruité.
-

8. Conclusion

Ce que les données disent vraiment

L'hypothèse initiale — "l'IA générative va accélérer le déclin des langages facilement générables" — n'est **pas supportée** par nos données. Le déclin d'un langage est prédit par des facteurs structurels anciens (âge, type, niveau d'abstraction), pas par son exposition à l'IA.

C'est un résultat scientifiquement intéressant *précisément parce qu'il est contre-intuitif*. Si l'IA générative avait un effet massif sur les choix de langages, on l'aurait détecté dans les données 2022-2024. Son absence suggère que :

- Les développeurs ne choisissent pas un langage en fonction de la capacité de l'IA à le générer
- L'écosystème (emplois, frameworks, communauté) compte plus que la "généralité"
- L'IA est un outil transversal qui bénéficie à *tous* les langages, pas un facteur de différenciation

Recommandations pratiques pour un dev en 2026

1. **Python** reste le choix le plus sûr : écosystème IA/ML, communauté massive, momentum positif.
2. **Rust, Go, TypeScript** : langages en forte croissance, soutenus par les GAFAM.
3. **Éviter de commencer par** : PHP, Perl, VB.NET, Delphi — en déclin structurel indépendamment de l'IA.
4. **L'IA comme outil, pas comme menace** : Apprenez à utiliser Copilot/Claude/ChatGPT *avec* votre langage, pas *à la place* de votre langage.

Pistes pour aller plus loin

- Intégrer les données de marché de l'emploi (LinkedIn, Indeed) comme source complémentaire
 - Analyser les pull requests GitHub par langage pour mesurer la part de code AI-generated
 - Attendre 2027-2028 pour avoir un recul suffisant sur l'effet LLM
 - Utiliser un dataset Twitter/Reddit spécifique aux développeurs pour le sentiment
-

Annexes

A. Features utilisées pour le modeling

Feature	Description
usage_pct	Pourcentage d'utilisation actuel
delta_1obs / 3obs / 6obs / 12obs	Variations sur différents horizons
ma_6, ma_12	Moyennes mobiles 6 et 12 mois
volatility_12	Écart-type glissant 12 mois
momentum_6	Tendance récente (dérivée)
rank	Rang par popularité
distance_to_peak	Distance relative au pic historique
months_since_peak	Mois écoulés depuis le pic
obs_count	Nombre d'observations cumulées
ai_susceptibility_score	Score hybride manuel+data (NON prédictif)
language_age	Âge du langage en années
is_static_typed	Typage statique (1) ou dynamique (0)
is_compiled	Compilé (1) ou interprété (0)
level_num	Niveau : low=1, mid=2, high=3
horizon_num	Horizon de prédiction (6/12/24 obs)
max_alltime	Usage maximal historique
n_observations	Nombre total d'observations pour ce langage

B. Hyperparamètres retenus

- **LogReg L2** (meilleur) : C=1.0, max_iter=1000, class_weight='balanced', solver='lbfgs'
- **XGBoost optimisé** : max_depth=5, n_estimators=200, lr=0.05, subsample=0.7, colsample_bytree=0.7 (CV F1=0.74, test F1=0.54 — overfitting)
- **LightGBM optimisé** : max_depth=5, n_estimators=200, lr=0.05, subsample=0.85, colsample_bytree=0.85, min_child_samples=50 (CV F1=0.74, test F1=0.57 — overfitting)

C. Figures

Figure	Description
fig_01	Trajectoires des 20 langages les plus populaires
fig_02	Comparaison pré/post LLM
fig_03	Corrélation SO vs GitHub
fig_04	Top 5 chutes / hausses 2022-2024
fig_05	Heatmap de volatilité
fig_06	Évolution des parts de marché (stacked area)
fig_07	Calibration du seuil de déclin
fig_08	Matrice de corrélation des features
fig_09	Matrice de confusion (LogReg L2)
fig_10	Importance par permutation
fig_11	Régression : prédit vs réel
fig_12	SHAP summary plot
fig_13	Analyse d'erreur par langage
fig_14	Quadrant de pertinence